

Final Exam for PHYS 512. Due before end of exam period.

There are two bonus points for following instructions, plus a bonus question, which can make up for mistakes elsewhere. All codes/work should be put into a zip/tar file and emailed to me. Do NOT put your answers on github. You may refer to notes/old codes and use google, but you may not consult anyone (including online) besides the TAs and the instructor. You may not consult LLMs such as chatgpt. There are three questions, plus some bonus points available. The final score is capped at 100. You have 24 hours to complete from when you start working.

Problem 0: (2 points)

Please put everything into a directory. Include a file called “myinfo.txt” that has your name and ID in it, so I can match grades to students at the end. Please do NOT put your name elsewhere in your codes/answers.

Problem 1: Is this a Polynomial? (20 points)

We saw that the n^{th} order Chebyshev polynomial is given by $T_n(x) = \cos(n \arccos(x))$ for x in $[-1, 1]$. It seems plausible that $\sin(n \arcsin(x))$ would also be polynomials. We’ll now investigate this. We’ll define $S_n(x) \equiv \sin(n \arcsin(x))$

Part A: (5 points)

Use an x-range of “`x=np.linspace(-1,1,2001)`”. Plot $S_n(x)$ for integer n between 0 and 5, inclusive (so 6 lines). Save this in “arcsin_plot.png”

Part B: (8 points)

If the S_n are n^{th} order polynomials, we should be able to express them in terms of the Chebyshev polynomials T_n . You can use `np.polynomial.chebyshev.chebvander` to generate the T_n if you don’t want to code up the recurrence relation. Do a least-squares fit you write yourself to find the best-fit coefficients when you fit S_n to T_0 through T_n using the x -range from above. Print these out for each S_n .

Part C: (7 points)

Which of these S_n (if any) are polynomials? Explain your reasoning.

Problem 2: Stability of Implicit Advection (40 points)

We’ll stick with the standard advection equation

$$\frac{\partial f(x, t)}{\partial t} = -v \frac{\partial f(x, t)}{\partial x}$$

Assume that v is positive for the rest of the problem. We saw when carrying out Von Neumann stability analysis of the advection equation that the

upwind derivative

$$\frac{\partial f(x, t)}{\partial x} = \frac{f(x, t) - f(x - dx, t)}{dx}$$

was stable as long as we satisfied the Courant–Friedrichs–Lewy (CFL) condition. The downwind derivative:

$$\frac{\partial f(x, t)}{\partial x} = \frac{f(x + dx, t) - f(x, t)}{dx}$$

was unconditionally unstable.

We also saw that implicit methods could introduce numerical diffusion that could restore stability to our problem. For real fluid codes that have velocities in both directions, we'd love for both the upwind and downwind derivatives to be stable so we don't have to check the fluid velocity before deciding how to take a derivative.

Part A: (8 points)

Consider the following implicit upwind derivative:

$$\frac{\partial f(x, t)}{\partial x} = \frac{f(x, t + dt) - f(x - dx, t + dt)}{dx}$$

(note we're evaluating the spatial derivative in the future now). That gives

$$\frac{f(x, t + dt) - f(x, t)}{dt} = -v \frac{f(x, t + dt) - f(x - dx, t + dt)}{dx}$$

Assuming $\alpha \equiv vdt/dx$, and $f(x, 0) = \exp(ikx)$, derive $f(x, dt)$. You should get something that looks like $\gamma \exp(ikx)$ where γ is a function of α, k, dx . What is your expression for γ ?

Part B: (8 points)

For what values of α (if any) are we stable?

Part C: (8 points)

Repeat parts A and B but using the downwind implicit derivative.

Part D: (12 points)

Implicit methods are usually a bit trickier to use because we need to use future values we do not yet know. This can usually be done by solving (very simple) matrix equations. If you do have a solution, it's easy to check that it is correct, though. In this part, we'll use your answer from part A to update our solution using an FFT. Write a routine that updates a solution

implicitly using Fourier space. Your routine should FFT the input data, work out the updated Fourier values given α and the number of time steps desired (default 1), and returns the data. Use the upwind derivative for this routine. The prototype should be

```
def update_fft(f,alpha,nstep=1):
```

Starting with initial conditions of a boxcar at $t = 0$,

```
f=np.zeros(300)
f[100:200]=1
```

and periodic boundary conditions, plot the *difference* between $f(x, t = dt)$ and $f(x, 0)$ after one timestep using your implicit solver. (note - one of the nice things about doing this via an FFT is that we can write the solution for any time without having to calculate the function at intermediate timesteps.)

Part E: (4 points)

Verify your solution is correct by plotting $f(x, dt) - f(x)$ against the implicit upwind derivative you can now work out with $f(x, dt)$ given your answer to Part D.

Problem 3: Compression of Audio Data (40 points)

We're going to use Kriging/linear prediction to see how you might compress audio data. The algorithm we're going to use here is very close to that used by FLAC, the standard for lossless audio compression (other formats such as mp3 lose information as part of their compression). Make sure scipy is installed, and you may wish to install sounddevice (but do not have to).

I have recorded a short (~ 10 second) audio sample from a keyboard set to organ mode and saved it in organ_sample.npy. The data in the file tell you the amplitude of the air pressure measured by a microphone as a function of time. Consequently, if you played a pure tone, you would expect to see a sinewave in the data.

Look at the file organ_code.py, which reads organ_sample.npy. If you have sounddevice installed and run the code, you should hear the sample play from your speakers. The code will also write a wavfile of the sample, which you should be able to listen to with any music player on your machine.

Part A: (5 points)

Do you think we can describe the organ sample as stationary? Why or why not.

Part B: (7 points)

Plot the power spectrum of the sample by taking its Fourier transform (since the data are real, please use `rfft` and not plain old `fft`). You should see a set of peaks. The lowest is called the fundamental, and the others are overtones. The overtones should appear at integer multiples of the fundamental. What is the k -value of the fundamental?

Part C: (7 points)

The sample rate is 44,100 samples per second (standard for audio). What is the total length of the sample, in seconds? We can use that to set the conversion between frequency (number of sine waves per second) and k from the FFT. What is the frequency, in Hz, of the fundamental?

Part D: (7 points)

If you were to plot a small piece of the data, you'd see you could do a pretty good job predicting the value of the next sample given some number of previous values. We saw how to do this using Kriging/linear prediction. The first step is to calculate a covariance matrix C where $C(i, j) = \langle d_i d_j \rangle$. For computational simplicity, we'll assume the data are stationary even if you think they weren't. In that case, the covariance reduces to a 1D function where $C(\delta) = \langle d_i d_{i+\delta} \rangle$. Calculate $C(\delta)$ for at least as many samples as your answer for part B plus one, divide by $C(0)$, and print out your first 10 values. If you are not sure of your answer to part B, you may use 150 (so C should be at least 151 numbers long).

Part E: (7 points)

Calculate the Kriging coefficients to predict the next sample given your chunk of samples. If you used 150 samples, then I want the coefficients to give the 151 sample given the previous 150 samples. Plot them in a plot called "kriging.coefficients.png"

Part F: (7 points) If you have say n coefficients, use your coefficients to predict the value of every sample past the first n . In all cases, use the true values for the prediction, and not the ones from your linear predictor. If you did not get Part E to work, I have included sample coefficients for a 125-element buffer in "coeffs_125.npy" which you can use. What is the median absolute value (`np.median(np.abs(r))`) where r is the difference between the original data and your reconstruction? Compare this to the median absolute value of the raw data (I get a value a bit under 9000 for the original data).

Bonus (Up to 5 point): If you want to see how well you did, write 16-bit versions of the original data and the difference between your prediction and

the data. You can write these as wavfiles using the code in the example script. Compress the wavfiles of the original version and the difference r . You can use any tool, but bzip2 works especially well. What is the ratio of sizes of the compressed difference file vs. the uncompressed original wavfile? The standard lore is that we can losslessly compress by a factor of 2. Did we get that here? If you actually wanted to play this, you would save the first n samples and the kriging coefficients. Then for every new sample, you would use the previous samples to predict its value, then look up the correction from the compressed file and add that in. That way you can exactly reconstruct the original data (at least, the 16-bit integer version of the original data) while only using the compressed difference file plus a little bit of extra information to get you started. For full bonus points, compare at least two difference compression methods.